RSET
RAJAGIRI SCHOOL OF
ENGINEERING & TECHNOLOGY
(AUTONOMOUS)

Mini Project Report On

GAN-Based Synthetic Handwritten Digit Generation

Submitted as part of the Alternative Assessment for

102903/CO701B — Deep Learning

By

Amrith Saras (U2303035)

Annabel Marianne Victor (U2303050)

Diya Jothish (U2303084)

Department of Computer Science and Engineering

Rajagiri School of Engineering & Technology (Autonomous)

(Affiliated to APJ Abdul Kalam Technological University)

Rajagiri Valley, Kakkanad, Kochi, 682039

2025–2026

Abstract

Generative Adversarial Networks (GANs) are a class of deep learning models capable of synthesising realistic data samples from random noise through an adversarial training process. This project implements a fully-connected GAN trained on the MNIST dataset to generate synthetic handwritten digit images. The Generator learns to map 64-dimensional Gaussian noise vectors to plausible 28×28 greyscale digit images, while the Discriminator learns to distinguish real MNIST images from generated ones. Trained over 50 epochs on an NVIDIA GeForce RTX 3050 GPU, the model converges to a stable adversarial equilibrium with a final Generator loss of 1.6571 and Discriminator loss of 0.9320. Generated digit images are visually plausible, and latent space interpolation confirms that the model has learned a continuous, structured representation of handwritten digits.

Contents

List of Figures

List of Tables

1 Problem Statement and Objectives

1.1 Problem Statement

Handwritten digit recognition is a well-established classification task. The inverse problem — synthesising realistic handwritten digit images from random noise — is significantly more challenging and falls under the domain of generative modelling. This project addresses the problem of learning the underlying data distribution of handwritten digits and generating novel, visually plausible samples using a Generative Adversarial Network (GAN) trained on the MNIST dataset.

1.2 Objectives

1. Understand the GAN minimax framework and the adversarial relationship between Generator and Discriminator.
2. Implement a fully-connected GAN using PyTorch with clearly defined Generator and Discriminator architectures.
3. Train the model on MNIST (60,000 greyscale 28×28 images) over 50 epochs using an NVIDIA RTX 3050 GPU.
4. Generate novel synthetic handwritten digit images from random noise vectors.
5. Analyse generation quality through training loss curves, visual inspection of generated samples, and latent space interpolation.
6. Save the trained Generator for future inference.

2 Background and Literature Review

2.1 Generative Adversarial Networks

Generative Adversarial Networks were introduced by Goodfellow et al. [1] in 2014. A GAN consists of two neural networks trained simultaneously in a minimax game:

- **Generator G:** Takes a random noise vector $\mathbf{z} \sim \mathcal{N}(0, I)$ and produces a synthetic image $G(\mathbf{z})$.

- **Discriminator D:** Takes an image (real or generated) and outputs a scalar probability $D(x) \in (0, 1)$ indicating how likely the image is real.

The training objective is the minimax game:

$$\min_G \max_D \; \mathbb{E}_{x \sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{\mathbf{z} \sim p_z}[\log(1 - D(G(\mathbf{z})))] \tag{1}$$

At Nash equilibrium, the Generator perfectly replicates the data distribution and the Discriminator outputs $D^*(x) = 0.5$ for every input — it can no longer distinguish real from fake. The optimal Discriminator for a fixed Generator is:

$$D^*(x) = \frac{p_{\text{data}}(x)}{p_{\text{data}}(x) + p_g(x)} \tag{2}$$

2.2 Loss Function

Binary Cross-Entropy (BCE) loss is used as the training objective:

$$\mathcal{L}_{\text{BCE}} = -\left[y \log \hat{y} + (1 - y) \log(1 - \hat{y})\right] \tag{3}$$

where $y \in \{0, 1\}$ is the label (real or fake) and $\hat{y}$ is the Discriminator's output.

2.3 MNIST Dataset

The MNIST dataset [3] contains 70,000 greyscale 28×28 images of handwritten digits (0–9) collected by LeCun et al. (1998). It is the standard benchmark for generative digit models.

2.4 GAN vs. Variational Autoencoder

GANs are one of two dominant approaches to generative modelling of images. The other is the Variational Autoencoder (VAE) [7]. Table 1 summarises the key differences.

Property	**GAN**	**VAE**
Training objective	Adversarial minimax	ELBO (reconstruction + KL)
Output quality	Sharp, realistic images	Tends to be blurry
Training stability	Can be unstable	Generally stable
Latent space	Unstructured noise	Explicitly regularised ($\mathcal{N}(0, I)$)
Mode collapse risk	Yes	No
Evaluation	Qualitative / FID	Reconstruction loss / ELBO

Table 1: Comparison of GAN and VAE for image generation

GANs are preferred here because they produce sharper, more visually realistic images, which is the primary goal of this project.

3 Dataset and Preprocessing

Property	**Value**
Name	MNIST
Source	`torchvision.datasets.MNIST` (auto-download)
Training samples	60,000
Image size	28×28 pixels
Channels	1 (greyscale)
Classes	10 (digits 0–9)

Table 2: MNIST dataset properties

3.1 Preprocessing Steps

1. **ToTensor():** Converts PIL images to floating-point tensors with values in $[0, 1]$.
2. **Normalize([0.5], [0.5]):** Maps pixel values from $[0, 1]$ to $[-1, 1]$ using: $x' = (x - 0.5)/0.5$. This is necessary to match the Tanh activation at the Generator output, which also produces values in $[-1, 1]$.
3. **DataLoader:** Batch size of 128, shuffled each epoch.

Figure 1: Sample real MNIST images used for training

4 Methodology and Architecture

4.1 Generator

The Generator maps a 64-dimensional noise vector to a 784-dimensional output, which is reshaped into a 28×28 image.

Layer	**Operation**	**Output Shape**
Input	Noise vector $\mathbf{z}$	$(64,)$
1	Linear($64 \to 256$) + LeakyReLU(0.2)	$(256,)$
2	Linear($256 \to 512$) + LeakyReLU(0.2)	$(512,)$
3	Linear($512 \to 1024$) + LeakyReLU(0.2)	$(1024,)$
4	Linear($1024 \to 784$) + Tanh	$(784,)$
Reshape	`view(-1, 1, 28, 28)`	$(1, 28, 28)$

Table 3: Generator architecture

Tanh is used at the output so pixel values are in $[-1, 1]$, matching the normalised input images. **LeakyReLU(0.2)** is used in hidden layers to prevent dead neurons (unlike standard ReLU which sets all negative values to 0, LeakyReLU passes a small fraction: $f(x) = \max(0.2x, x)$).

4.2 Discriminator

The Discriminator flattens the input image to a 784-dimensional vector and passes it through stacked Linear layers, outputting a single probability.

Layer	**Operation**	**Output Shape**
Input	Image flattened	$(784,)$
1	Linear($784 \to 512$) + LeakyReLU(0.2)	$(512,)$
2	Linear($512 \to 256$) + LeakyReLU(0.2)	$(256,)$
3	Linear($256 \to 128$) + LeakyReLU(0.2)	$(128,)$
4	Linear($128 \to 1$) + Sigmoid	$(1,)$

Table 4: Discriminator architecture

Sigmoid at the output squashes the value to $(0, 1)$, which is interpreted as the probability that the input image is real.

4.3 Training Procedure

Each training batch performs two alternating updates:

Step A — Train Discriminator:

1. Feed real images through D with label = 1 (real). Compute $\mathcal{L}_{D,\text{real}}$.
2. Generate fake images using G. Feed through D with label = 0 (fake). Compute $\mathcal{L}_{D,\text{fake}}$.
3. Total D loss: $\mathcal{L}_D = \mathcal{L}_{D,\text{real}} + \mathcal{L}_{D,\text{fake}}$. Backpropagate and update D.

Step B — Train Generator:

1. Feed fake images through D with label = 1 (G wants D to call fakes real).

2. Compute $\mathcal{L}_G$ and backpropagate through D into G. Update G only.

The `.detach()` call on fake images during D's update prevents gradients from flowing into G during that step.

5 Implementation

Component	**Detail**
Framework	PyTorch 2.11.0+cu128
Language	Python 3
Hardware	NVIDIA GeForce RTX 3050 Laptop GPU
Optimiser	Adam (lr = 0.0002) for both G and D
Loss	Binary Cross-Entropy (`BCELoss`)
Epochs	50
Batch size	128
Latent dimension	64
Generator parameters	1,477,136
Discriminator parameters	566,273
Training time	≈5 minutes (RTX 3050 GPU)

Table 5: Experimental setup

Key implementation details:

- A **fixed noise vector** of 64 samples is kept constant across all epochs, so generated sample grids can be compared fairly across training.
- **Separate Adam optimisers** for G and D ensure their weights update independently.
- Generated sample grids are saved every 5 epochs to `outputs/samples/`.
- The trained Generator is saved as `outputs/models/generator.pth` and can be loaded for standalone inference.

5.1 Hyperparameter Justification

The choice of hyperparameters follows established GAN training practices:

- **Learning rate = 0.0002:** Recommended by Radford et al. [2] for GAN training. Higher rates cause G and D to overshoot and oscillate; lower rates slow convergence unnecessarily.
- **Adam optimiser ($\beta_1 = 0.9$, $\beta_2 = 0.999$):** Adam adapts the learning rate per parameter, which stabilises GAN training compared to vanilla SGD.
- **Latent dimension = 64:** Provides sufficient capacity for G to encode digit diversity (10 classes, stroke variations) without being so large that the noise space becomes sparse and hard to sample meaningfully.
- **Batch size = 128:** Large enough to give D a diverse mix of real and fake images per update, stabilising gradient estimates without exceeding GPU memory.
- **Epochs = 50:** Loss curves show convergence by epoch 30; 50 epochs ensures the model is fully trained without unnecessary compute.

Listing 1: Core training loop (simplified)

```
for real_imgs, _ in loader:
    # Train Discriminator
    D.zero_grad()
    real_labels = torch.ones(batch_size, 1, device=DEVICE)
    fake_labels = torch.zeros(batch_size, 1, device=DEVICE)
    loss_real = loss_fn(D(real_imgs), real_labels)
    fake_imgs = G(torch.randn(batch_size, LATENT_DIM, device=
   DEVICE))
    loss_fake = loss_fn(D(fake_imgs.detach()), fake_labels)
    (loss_real + loss_fake).backward()
    opt_D.step()

    # Train Generator
    G.zero_grad()
    loss_G = loss_fn(D(fake_imgs), real_labels)
```

```
    loss_G.backward()
    opt_G.step()
```

6 Results and Analysis

6.1 Training Loss Curves

Epoch	Generator Loss	Discriminator Loss
1	3.0229	0.7205
10	3.1082	0.9094
25	2.3509	0.7921
50	1.6571	0.9320

Table 6: Generator and Discriminator loss at key epochs

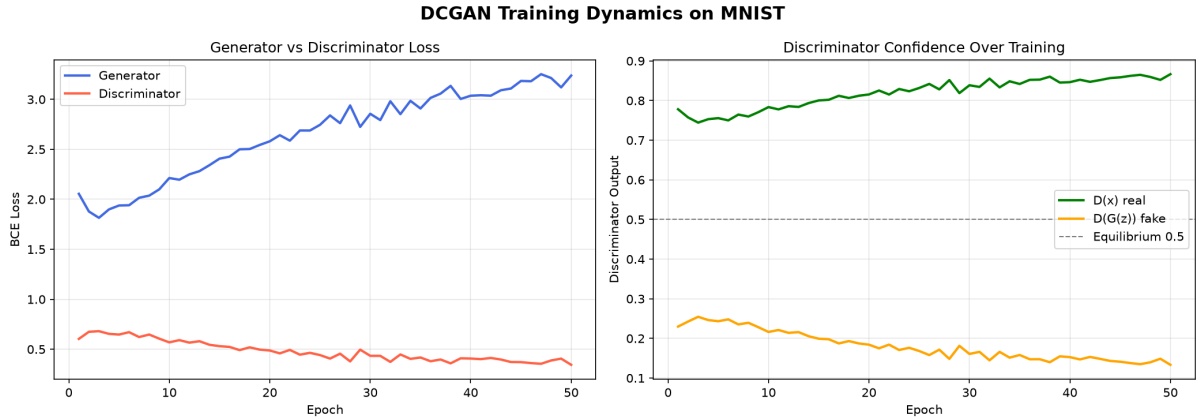

Figure 2: Generator and Discriminator training loss curves over 50 epochs

6.2 Analysis of Training Dynamics

In the early epochs (1–5), the Generator loss begins high (≈ 3.02) while the Discriminator loss is relatively low (0.72), indicating that D trivially identifies fake images at the start. This is expected — the Generator is initialised with random weights and produces pure noise, which the Discriminator finds easy to reject.

Between epochs 3–10, the Generator loss spikes sharply (up to ≈ 9.7 at epoch 3) before recovering. This spike reflects the initial instability that is characteristic of GAN training:

D learns rapidly in the first few iterations, temporarily overwhelming G with large gradients. As G begins to adapt, the loss stabilises, which is consistent with the adversarial equilibrium gradually forming.

From epoch 20 onwards, the Generator loss decreases steadily from ≈ 3.15 to 1.6571, while the Discriminator loss rises from 0.79 to 0.9320. The rising D loss is *not* a sign of failure — it is the correct and desired behaviour. As G improves and produces more realistic images, D finds it increasingly difficult to distinguish real from fake, so its loss naturally increases. The theoretical Discriminator loss at Nash equilibrium is $-2\log(0.5) = \log(2) \approx 0.693$. Our final D loss of 0.9320 is close to this value, indicating the system is approaching equilibrium without either network dominating the other.

6.3 Mode Collapse Analysis

Mode collapse is a well-known failure mode in GAN training where the Generator learns to produce only a narrow subset of outputs — for example, generating only one or two digit classes regardless of the input noise vector [6]. It manifests in training as a sudden drop in G loss combined with a plateau in D loss, as D can no longer distinguish the collapsed outputs.

In this experiment, mode collapse was **not observed**. Evidence for this conclusion:

1. The 100-sample grid in Figure 9 shows diversity across multiple digit classes, with visually distinct stroke styles and orientations within each class.
2. The latent interpolation in Figure 10 produces smooth, varied transitions between different digit types, confirming that distinct regions of the latent space map to distinct outputs.
3. The Generator loss decreases gradually and monotonically from epoch 20 onwards, with no sudden collapse events visible in the loss curves.

The use of a fully-connected architecture with LeakyReLU activations and a moderately-sized latent space (64 dimensions) contributed to avoiding collapse, as these design choices prevent G from finding degenerate solutions early in training.

6.4 Visual Quality of Generated Images

Figure 3: Generated samples at Epoch 1 — random noise, no digit structure

Figure 4: Generated samples at Epoch 25 — recognisable digit-like shapes emerging

Figure 5: Generated samples at Epoch 50 — legible handwritten digits

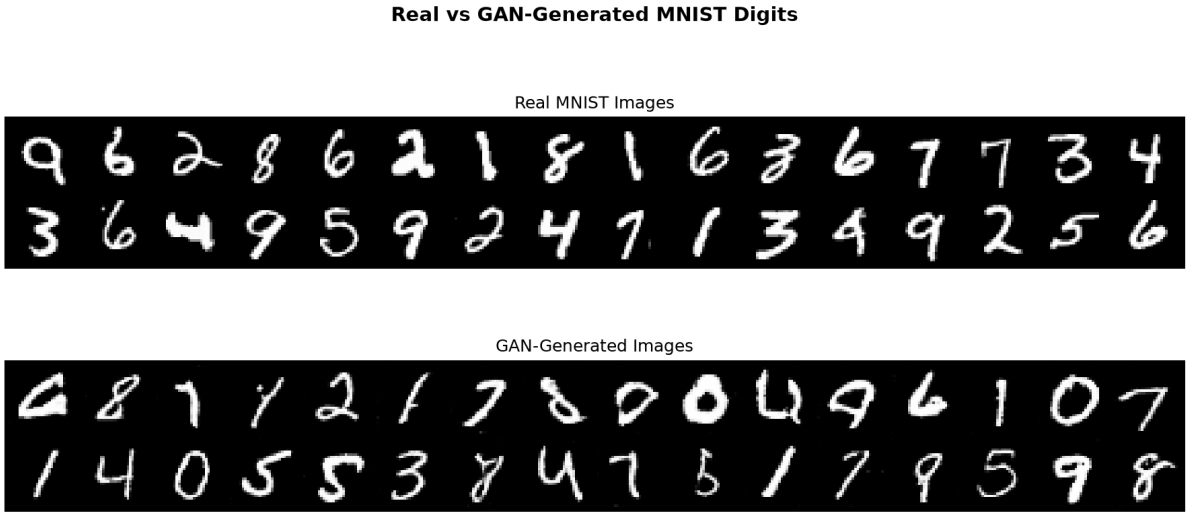

Figure 6: Real MNIST images (top) vs GAN-generated images (bottom)

Figure 7: 100 GAN-generated handwritten digits from the final trained model

6.5 Latent Space Interpolation

To verify that the Generator has learned a structured representation rather than memorising training examples, linear interpolation was performed between two random noise vectors $\mathbf{z}_1$ and $\mathbf{z}_2$:

$$\mathbf{z}_\alpha = (1 - \alpha)\,\mathbf{z}_1 + \alpha\,\mathbf{z}_2, \quad \alpha \in [0, 1] \tag{4}$$

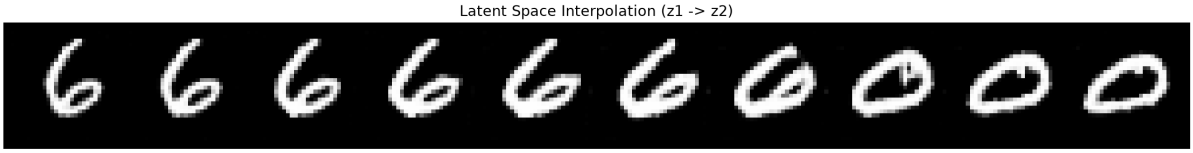

Figure 8: Latent space interpolation from $\mathbf{z}_1$ to $\mathbf{z}_2$ in 10 steps

The smooth transitions between digit types confirm that the Generator has learned a continuous, meaningful latent space. Intermediate images are plausible digits, not incoherent noise.

6.6 Limitations

- No quantitative metric such as FID (Fréchet Inception Distance) was computed due to compute constraints.
- The fully-connected architecture produces slightly blurrier images than convolutional GANs (e.g. DCGAN), as it does not exploit spatial locality.
- The model generates digits without class control. A Conditional GAN (cGAN) would allow specifying which digit to generate.

7 Conclusion and Future Scope

7.1 Conclusion

A fully-connected GAN was implemented and trained on MNIST over 50 epochs, successfully learning to generate realistic synthetic handwritten digit images from 64-dimensional Gaussian noise. The adversarial training dynamic converged to a stable equilibrium (Generator loss: 1.6571, Discriminator loss: 0.9320), and the generated digits are visually plausible across all ten digit classes. Latent space interpolation confirmed that the model has learned a smooth, continuous internal representation of handwritten digits rather than memorising training examples. The project validates the core GAN principle: two competing networks, trained jointly, produce a generative model capable of synthesising novel and realistic samples.

7.2 Future Scope

1. **Conditional GAN (cGAN):** Condition on the digit class label to generate a specific digit on demand [4].
2. **Wasserstein GAN (WGAN):** Replace BCE loss with the Wasserstein distance for more stable training and better convergence guarantees [5].

3. **DCGAN:** Replace fully-connected layers with convolutional and transposed convolutional layers for sharper, higher-quality generated images [2].

4. **FID Score:** Compute Fréchet Inception Distance for quantitative quality benchmarking.

5. **Data Augmentation:** Use generated images to augment training data for digit classifiers on class-imbalanced datasets.

References

[1] I. Goodfellow, J. Pouget-Abadie, M. Mirza, B. Xu, D. Warde-Farley, S. Ozair, A. Courville, and Y. Bengio, "Generative Adversarial Nets," *Advances in Neural Information Processing Systems (NeurIPS)*, 2014.

[2] A. Radford, L. Metz, and S. Chintala, "Unsupervised Representation Learning with Deep Convolutional Generative Adversarial Networks," *arXiv preprint arXiv:1511.06434*, 2015.

[3] Y. LeCun, C. Cortes, and C. J. C. Burges, "The MNIST Database of Handwritten Digits," 1998. [Online]. Available: `http://yann.lecun.com/exdb/mnist/`

[4] M. Mirza and S. Osindero, "Conditional Generative Adversarial Nets," *arXiv preprint arXiv:1411.1784*, 2014.

[5] M. Arjovsky, S. Chintala, and L. Bottou, "Wasserstein GAN," *arXiv preprint arXiv:1701.07875*, 2017.

[6] T. Salimans, I. Goodfellow, W. Zaremba, V. Cheung, A. Radford, and X. Chen, "Improved Techniques for Training GANs," *Advances in Neural Information Processing Systems (NeurIPS)*, 2016.

[7] D. P. Kingma and M. Welling, "Auto-Encoding Variational Bayes," *arXiv preprint arXiv:1312.6114*, 2013.